Kidney disease PCA

This notebook aims to perform **Principal Component Analysis (PCA)** on a dataset related to Chronic Kidney Disease (CKD) to reduce its complexity and then build and evaluate various machine learning models to classify patients as having or not having CKD.

Here's a breakdown of the process:

1. **Data Loading**: Getting the data into our workspace.
2. **Initial Data Exploration**: Looking at the data to understand its structure, find missing pieces, and get a general idea of what it contains.
3. **Data Visualization**: Creating charts and graphs to visually understand patterns and relationships in the data.
4. **Data Preprocessing**: Cleaning and preparing the data for machine learning, which often involves handling missing values, converting text data into numbers, and scaling numbers so they are all on a similar range.
5. **Principal Component Analysis (PCA)**: A technique to simplify the data by finding the most important patterns or components.
6. **Model Training**: Using the prepared data to teach different machine learning models how to predict kidney disease.
7. **Model Evaluation**: Checking how well these models perform.
8. **Model Comparison**: Deciding which model is the best for this task.

https://www.kaggle.com/code/answer3/kidney-disease-pca

1. Data Loading and Initial Setup

First, we need to load the data. Think of data loading like opening a spreadsheet in a program like Excel. We use a library called `pandas` in Python, which is excellent for working with tabular data (like spreadsheets). We also import `matplotlib` and `seaborn` for creating visualizations.

- `import pandas as pd`: Brings in the `pandas` library, giving it a shorter name `pd` for convenience.
- `import matplotlib.pyplot as plt`: Brings in `matplotlib` for plotting, `plt` is its common alias.
- `import seaborn as sns`: Brings in `seaborn`, which helps make plots look nicer and adds more plotting capabilities.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

The next step reads the kidney disease data from a file named `kidney_disease.csv`. The `na_values=["\t?",'?']` part tells pandas to treat `"\t?"` and `'?'` (which might be used in the dataset to represent missing information) as actual missing values (often shown as `NaN` or `Not a Number`).

`df.head()` then shows the first 5 rows of this loaded data, which gives us a quick peek at what the data looks like, including the column names and some initial values.

```
df = pd.read_csv('/content/drive/MyDrive/Chronic KIdney Disease 1/kid
df.head()
```

	id	age	bp	sg	al	su	rbc	pc	pcc	ba	...	
0	0	48.0	80.0	1.020	1.0	0.0	NaN	normal	notpresent	notpresent	...	4
1	1	7.0	50.0	1.020	4.0	0.0	NaN	normal	notpresent	notpresent	...	3
2	2	62.0	80.0	1.010	2.0	3.0	normal	normal	notpresent	notpresent	...	3
3	3	48.0	70.0	1.005	4.0	0.0	normal	abnormal	present	notpresent	...	3
4	4	51.0	80.0	1.010	2.0	0.0	normal	normal	notpresent	notpresent	...	3

5 rows × 26 columns

2. Initial Data Exploration: Understanding Our Data

After loading the data, it's crucial to understand what's inside. This is like looking at the summaries of each column in your spreadsheet.

- `df.describe()`: This command provides a statistical summary of all the *numerical* columns in our dataset. For each column, it shows things like:
  - `count`: How many non-missing values are there.
  - `mean`: The average value.
  - `std`: The spread of the values (standard deviation).
  - `min` / `max`: The smallest and largest values.
  - `25%`, `50%`, `75%`: Quartiles, which help understand the distribution of data (e.g., 50% is the median).

This helps us quickly spot potential issues like very wide ranges, or if a column has many fewer entries than others (suggesting lots of missing data).

```
df.describe()
```

	id	age	bp	sg	al	su	
count	400.000000	391.000000	388.000000	353.000000	354.000000	351.000000	3
mean	199.500000	51.483376	76.469072	1.017408	1.016949	0.450142	1
std	115.614301	17.169714	13.683637	0.005717	1.352679	1.099191	
min	0.000000	2.000000	50.000000	1.005000	0.000000	0.000000	
25%	99.750000	42.000000	70.000000	1.010000	0.000000	0.000000	
50%	199.500000	55.000000	80.000000	1.020000	0.000000	0.000000	1
75%	299.250000	64.500000	80.000000	1.020000	2.000000	0.000000	1
max	399.000000	90.000000	180.000000	1.025000	5.000000	5.000000	4

After getting a summary of numerical values, we also need to explicitly check for missing values across *all* columns (both numerical and text-based).

- `df.isnull().sum()`: This counts how many missing values (`NaN`) are in each column.
- `.to_frame(name='Missing')`: Converts these counts into a small table with a column named 'Missing'.
- `missing_info['% Missing'] = ...`: Calculates the percentage of missing values for each column.
- `missing_info = missing_info[missing_info['Missing'] > 0].sort_values(...)`: This filters the table to only show columns that *have* missing values and then sorts them from most missing to least missing.

Understanding missing data is very important because machine learning models often cannot work directly with missing values, and we'll need to decide how to handle them later (e.g., fill them in or remove the columns/rows).

```
missing_info = df.isnull().sum().to_frame(name='Missing')
missing_info['% Missing'] = (missing_info['Missing'] / len(df) * 100)
missing_info = missing_info[missing_info['Missing'] > 0].sort_values(
print(missing_info)
```

```
       Missing  % Missing
rbc        152      38.00
rc         131      32.75
wc         106      26.50
pot         88      22.00
sod         87      21.75
pcv         71      17.75
pc          65      16.25
hemo        52      13.00
su          49      12.25
sg          47      11.75
```

```
al          46      11.50
bgr         44      11.00
bu          19       4.75
sc          17       4.25
bp          12       3.00
age          9       2.25
pcc          4       1.00
ba           4       1.00
htn          2       0.50
dm           2       0.50
cad          2       0.50
appet        1       0.25
pe           1       0.25
ane          1       0.25
```

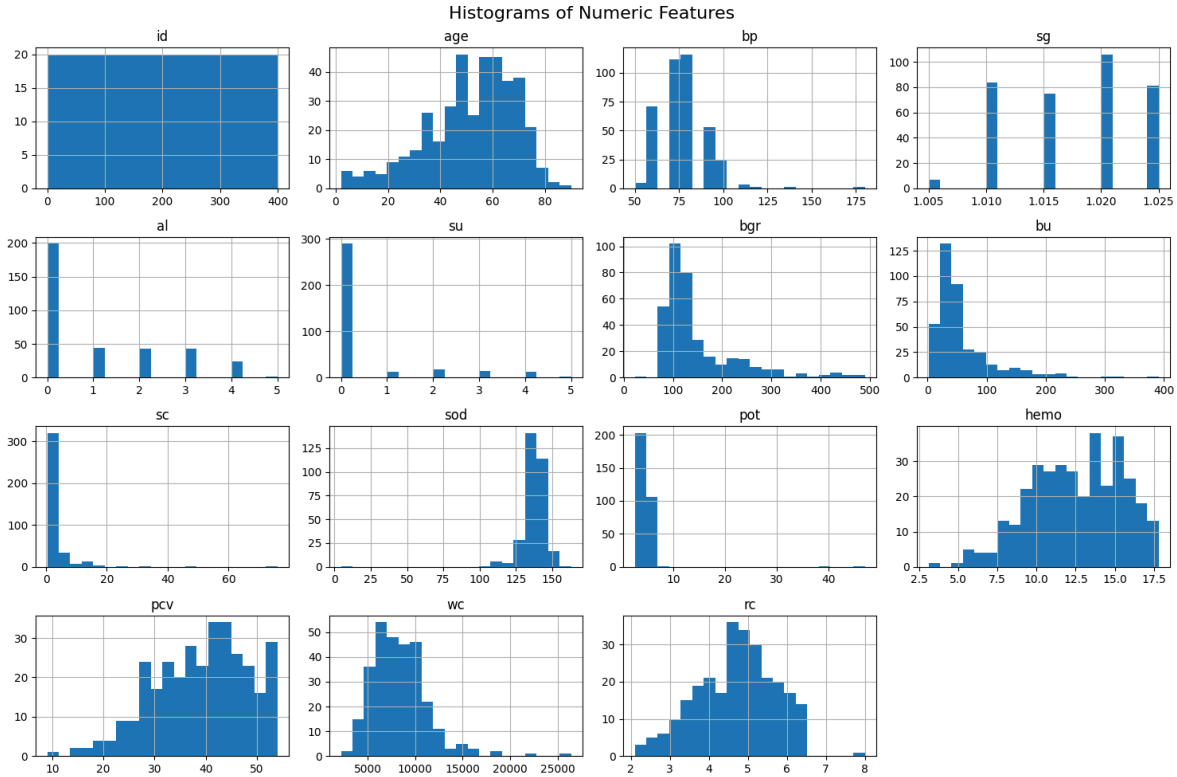
3. Data Visualization: Seeing the Data

Visualizations help us understand the data much better than just looking at numbers. They are like drawing pictures from our spreadsheet data.

- `df.select_dtypes(include='number').hist(...)`: This creates a histogram for every numerical column in the dataset. A histogram shows the distribution of values for a single variable—how often different values appear.
  - `figsize=(15, 10)`: Sets the size of the overall plot.
  - `bins=20`: Divides the data into 20 segments for the histogram bars.
  - `plt.suptitle(...)`: Adds a main title to the entire set of histograms.
  - `plt.tight_layout()`: Adjusts the plots so they don't overlap.
  - `plt.show()`: Displays the plots.

These histograms help us see if data is spread out evenly, clustered in certain areas, or skewed.

```
df.select_dtypes(include='number').hist(figsize=(15, 10), bins=20)
plt.suptitle("Histograms of Numeric Features", fontsize=16)
plt.tight_layout()
plt.show()
```



Next, we create box plots to see how each numerical feature relates to the `classification` column (which tells us if someone has kidney disease or not). A box plot is a way to visualize the distribution of data based on a five-number summary: minimum, first quartile, median, third quartile, and maximum.

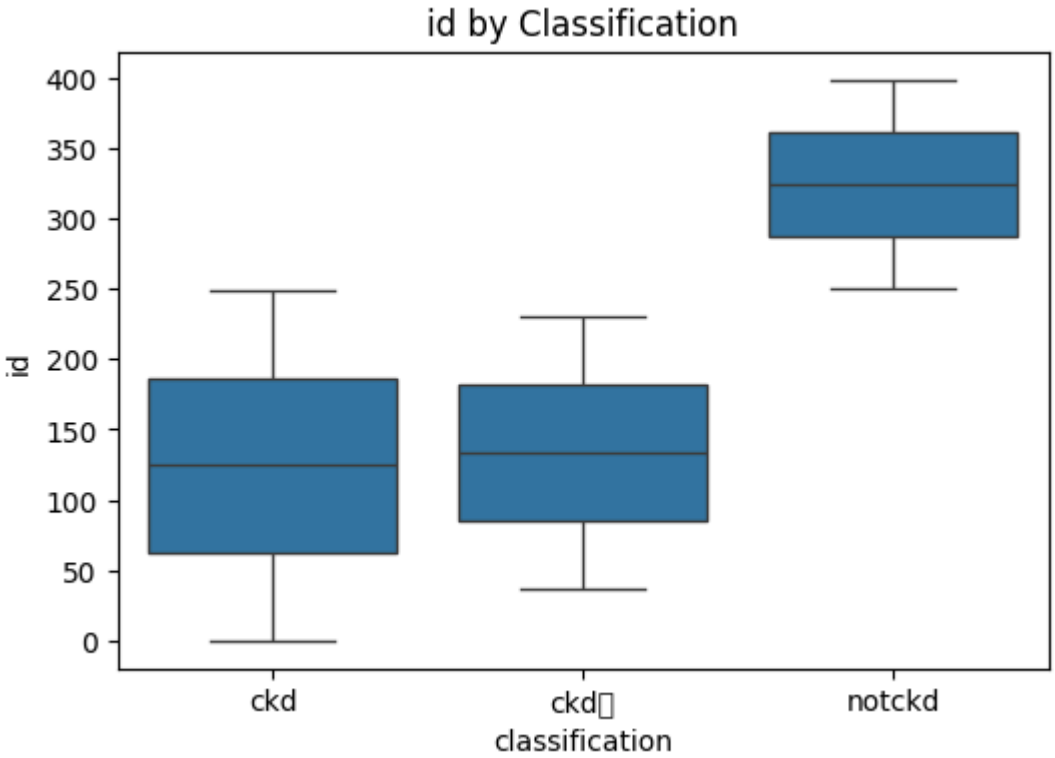
- `for col in numerics:` This loop goes through each numerical column.

- `sns.boxplot(data=df, x='classification', y=col)`: For each numerical column, it creates a box plot. The `x` axis is the 'classification' (kidney disease or not), and the `y` axis is the numerical column's values. This helps us visualize if there are noticeable differences in the distribution of these measurements between people with and without kidney disease. For example, if people with CKD tend to have much higher or lower values for a certain measurement, it would be visible in these plots.

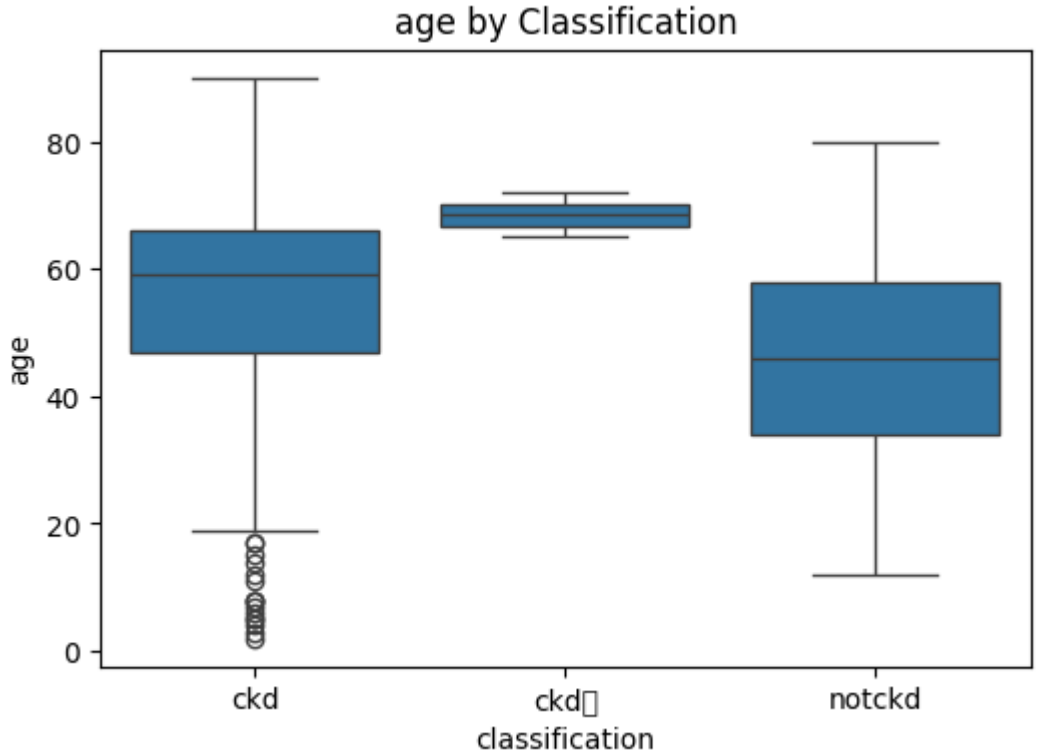
The error message you saw (`UserWarning: Glyph 9 (\t) missing from font(s) DejaVu Sans.`) is generally harmless and means that some special character (like a tab, represented by `\t`) was found in the text being plotted, and the chosen font couldn't display it. It doesn't prevent the plots from being generated correctly. It likely comes from the `na_values` replacement earlier. You might notice an empty plot due to the error, indicating the original box plots did not render properly for some features. We will fix this during the data preprocessing step.

```
numerics = df.select_dtypes(include='number').columns
for col in numerics:
    plt.figure(figsize=(6, 4))
    sns.boxplot(data=df, x='classification', y=col)
    plt.title(f'{col} by Classification')
    plt.show()
```

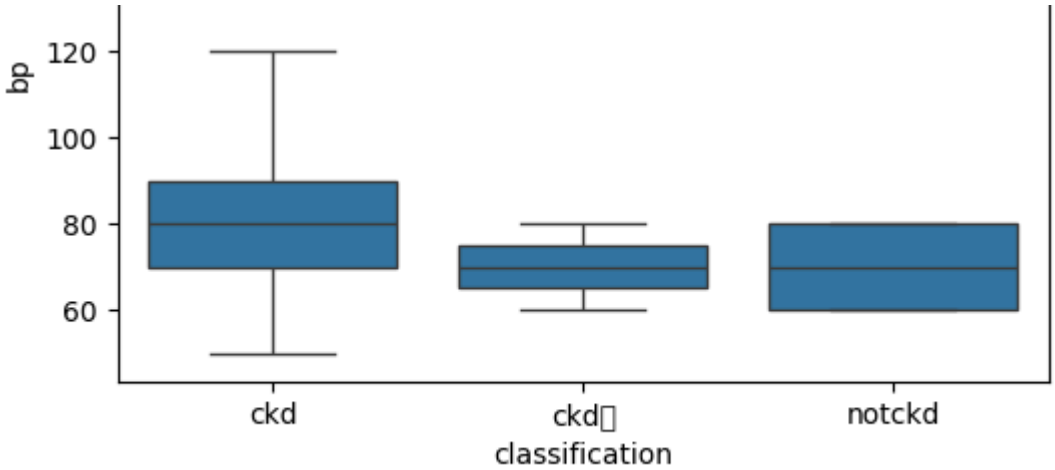
```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```



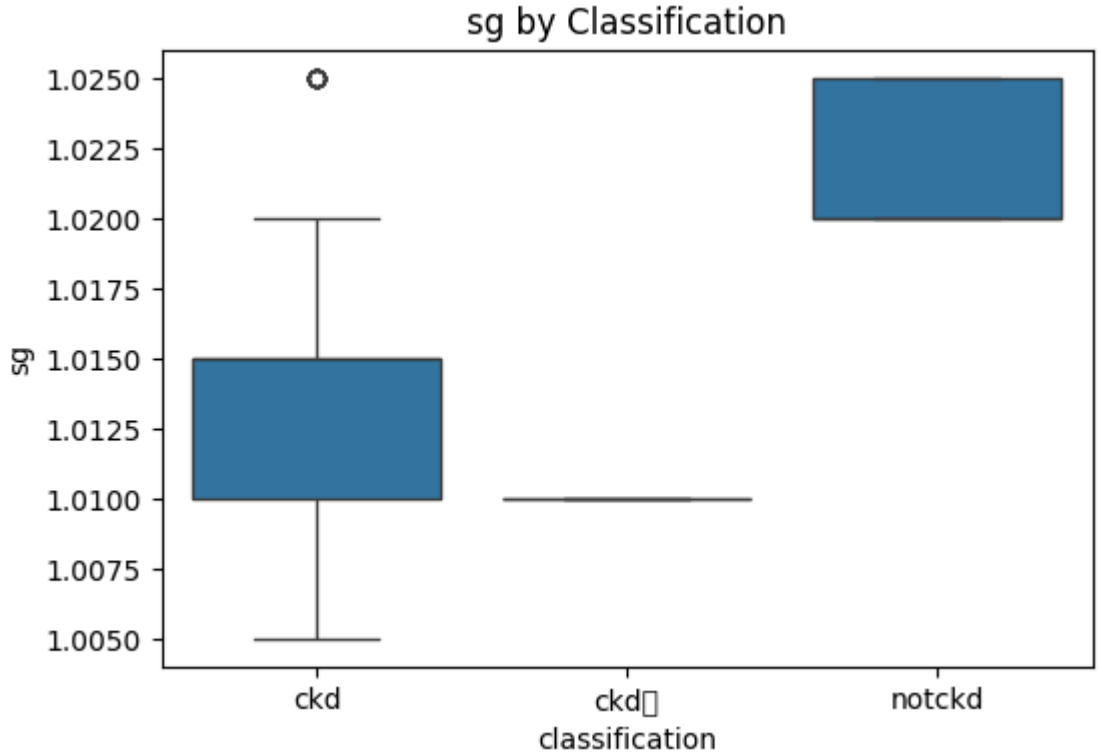

```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```



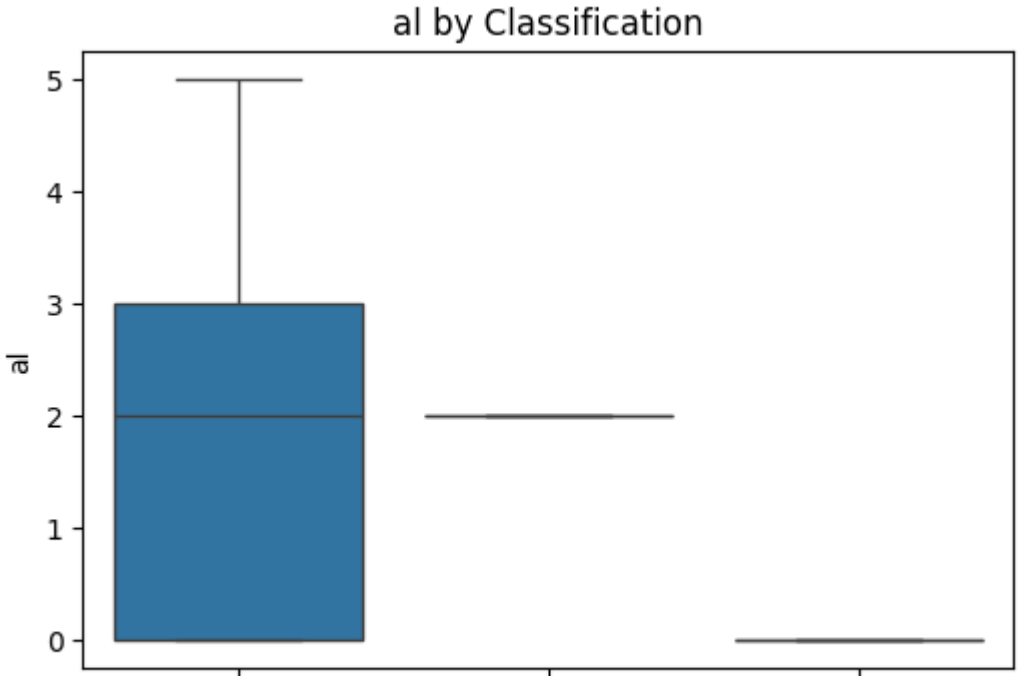

```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```

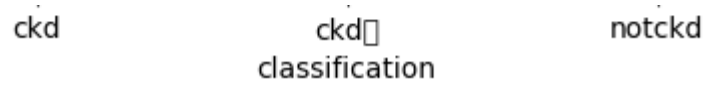



```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```

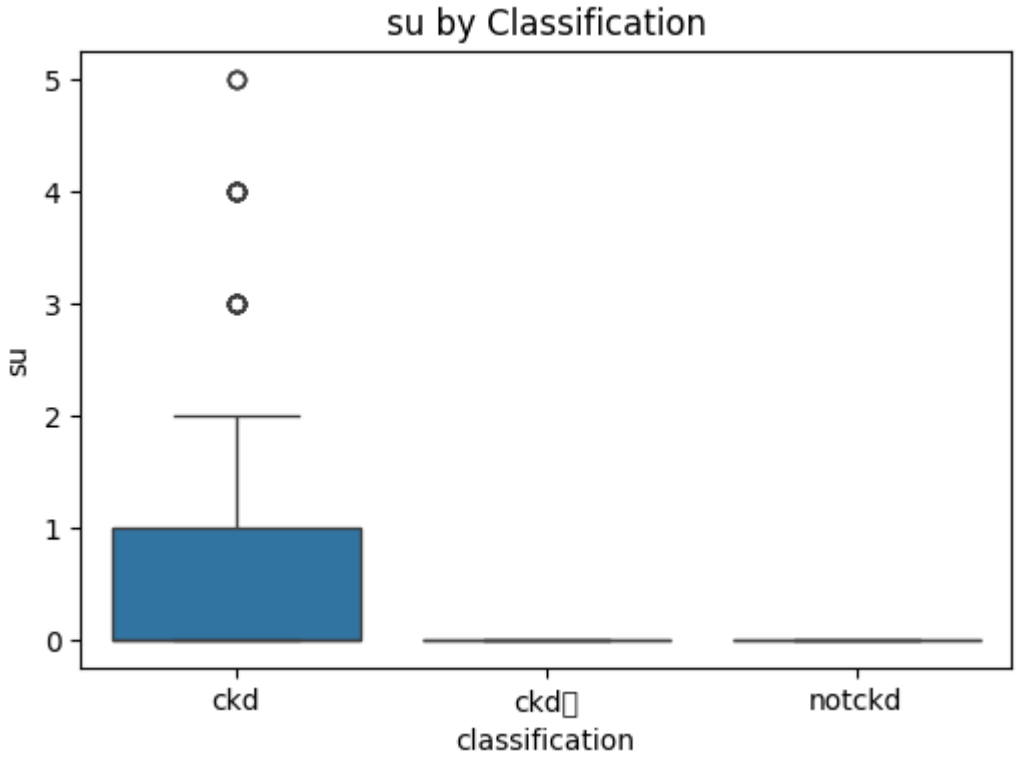



```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```

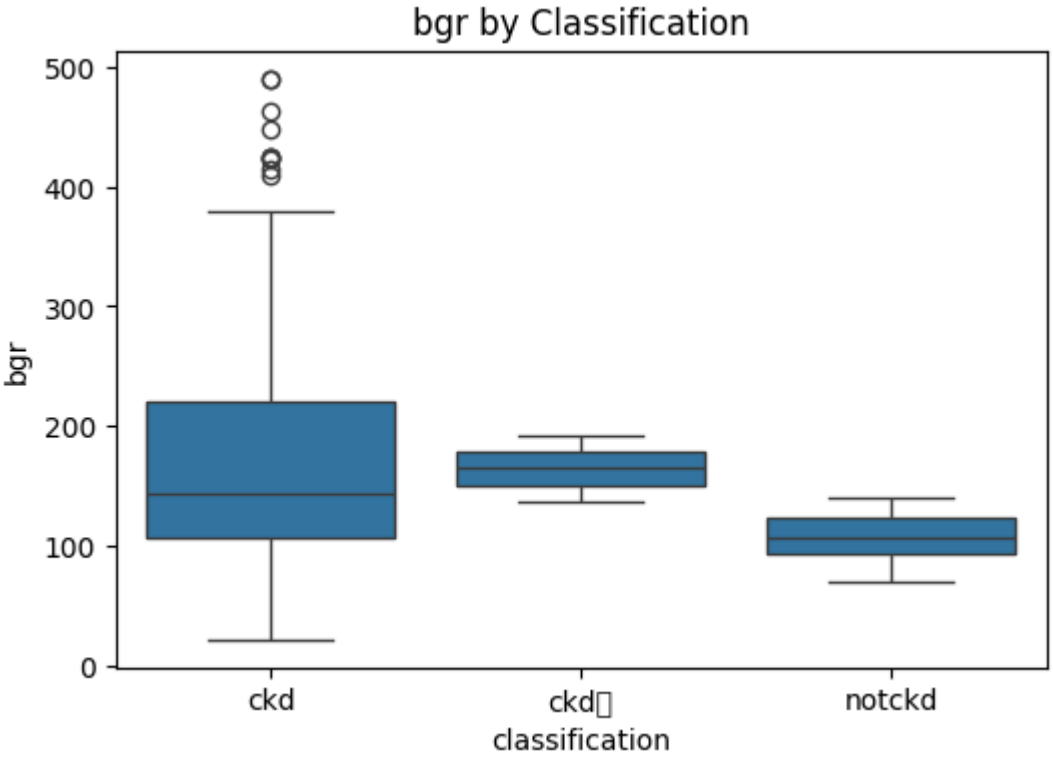




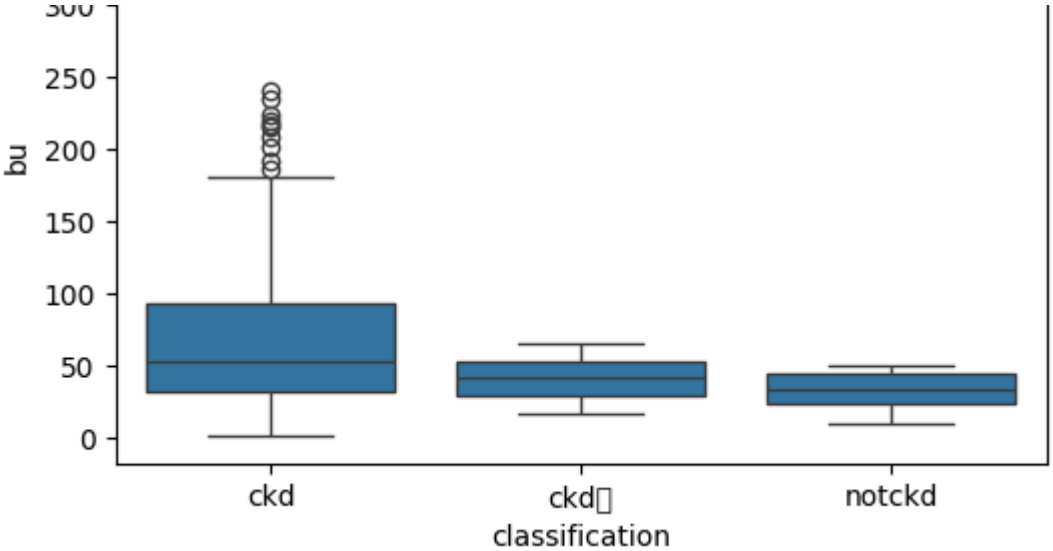

```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```



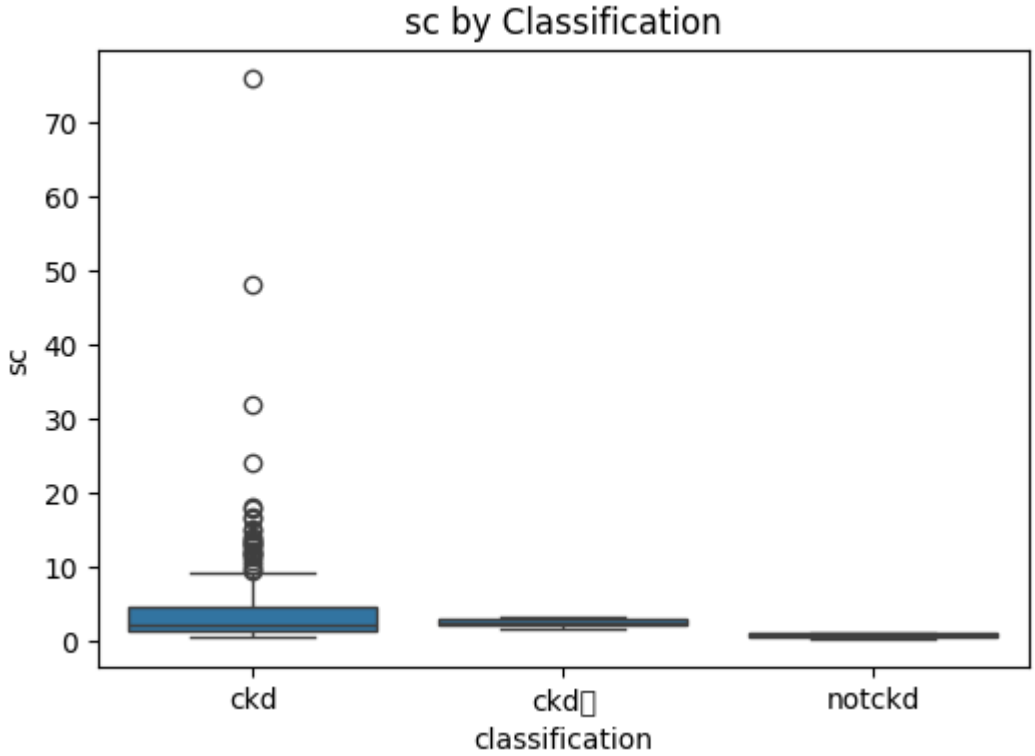

```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```



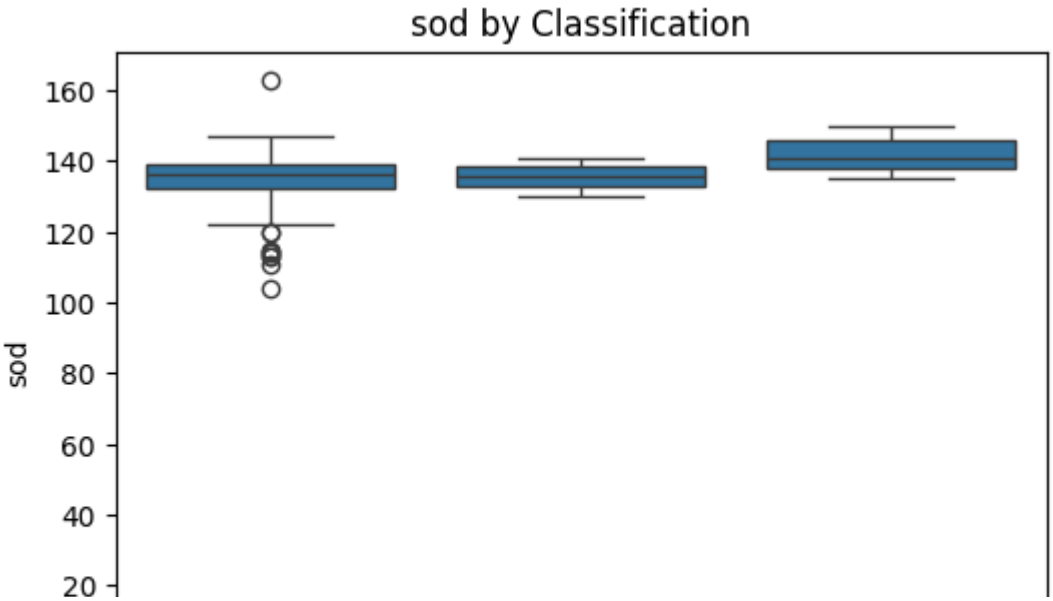

```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```

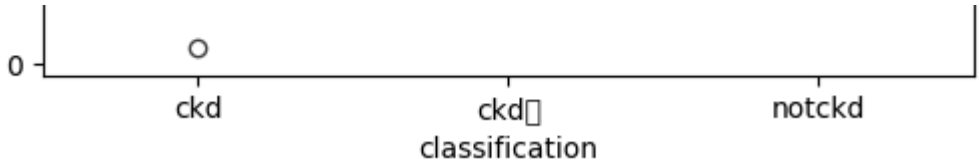



```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```

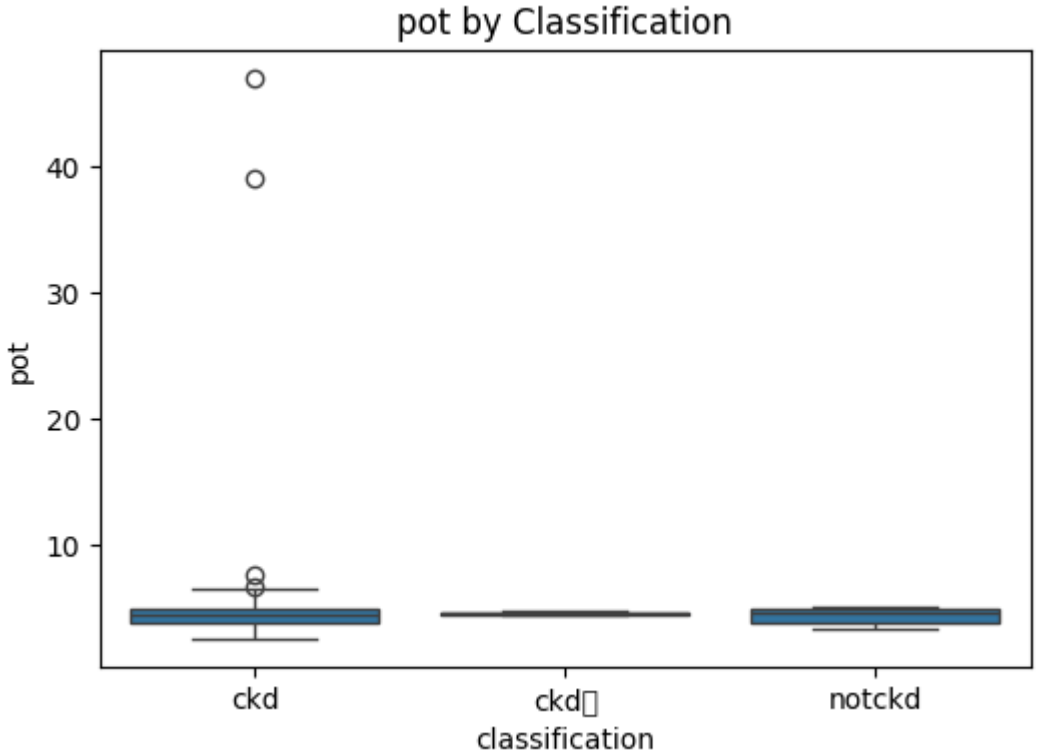



```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```

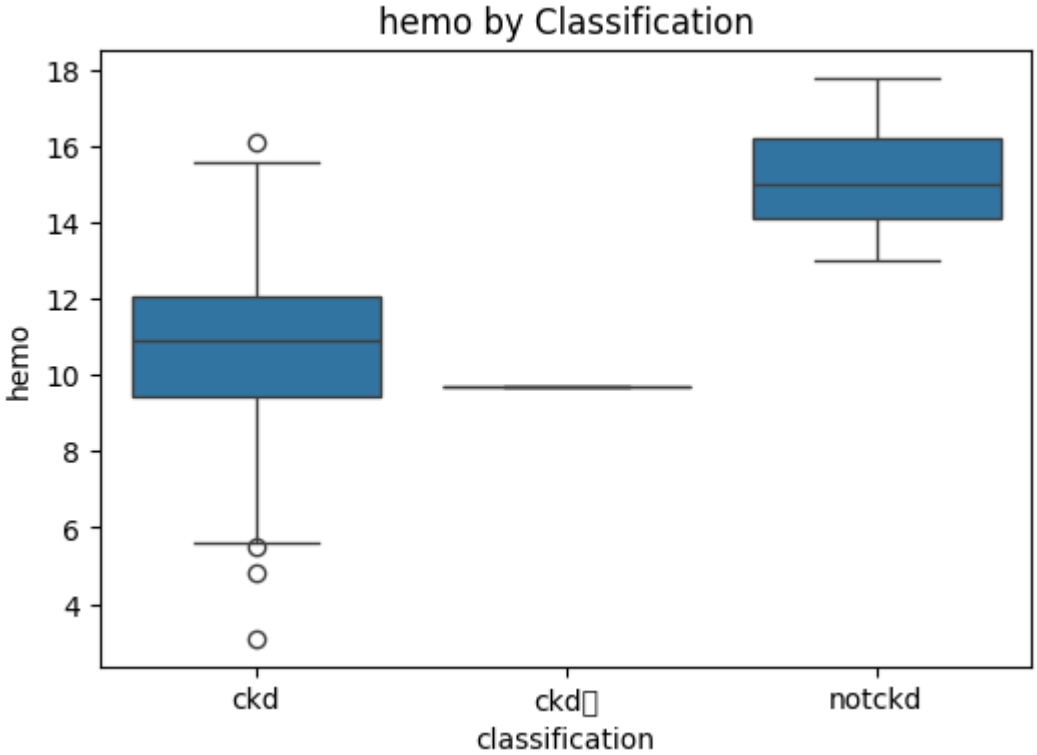




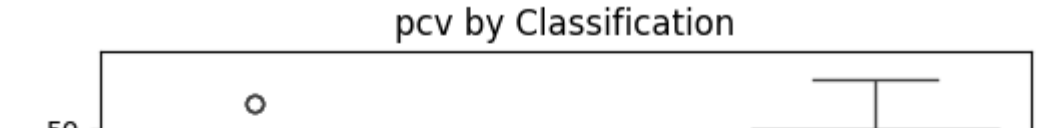

```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```

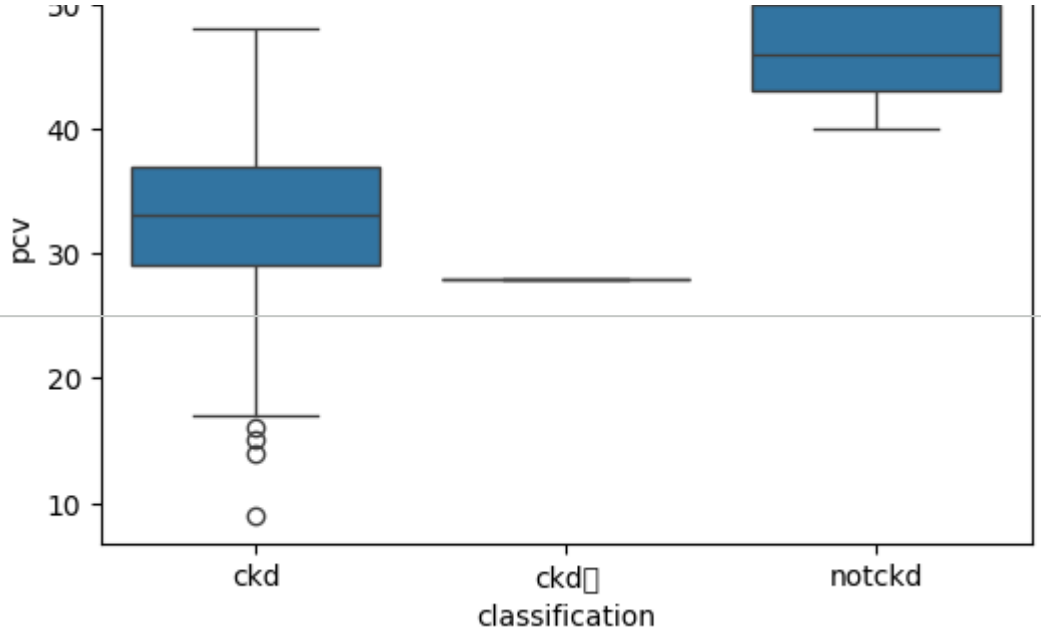



```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```

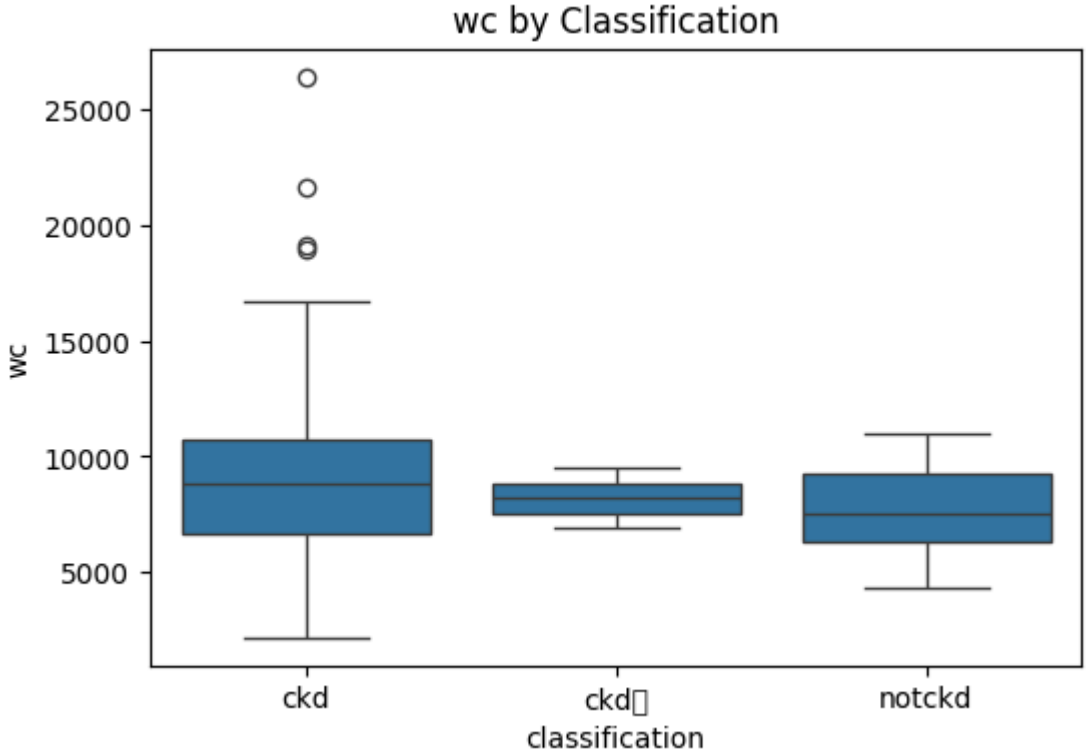



```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```

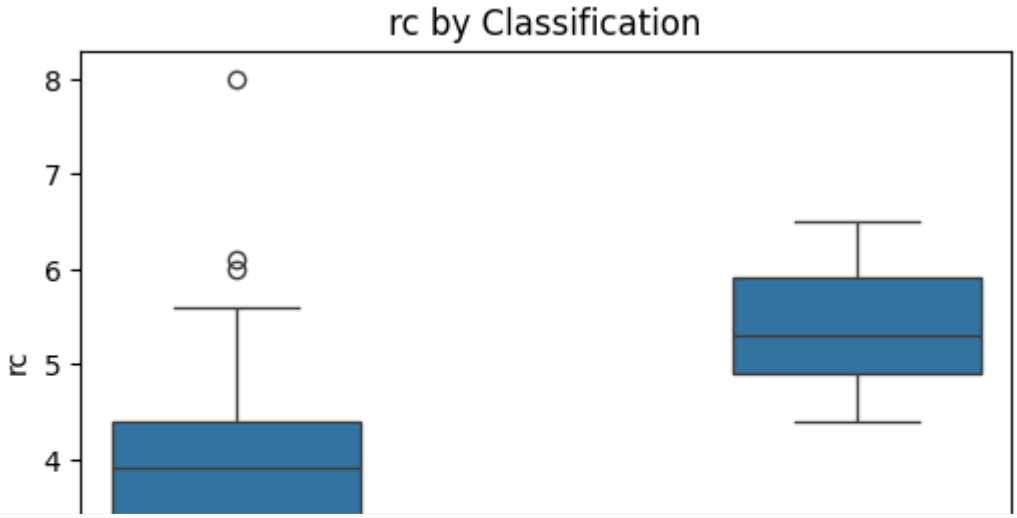





```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```




```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:15
  fig.canvas.print_figure(bytes_io, **kw)
```



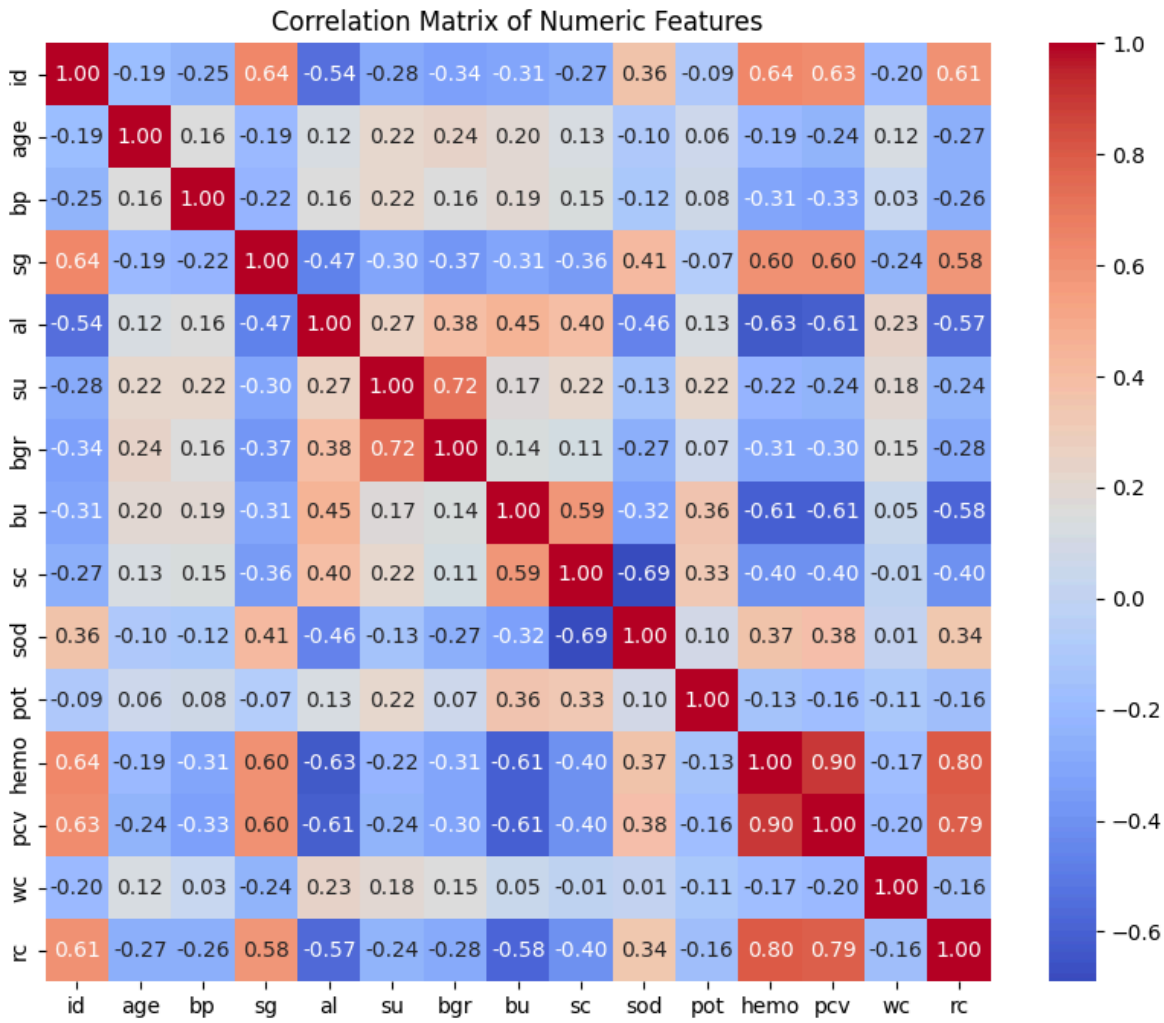
4. Data Preprocessing: Getting Ready for Machine Learning

Before we can train machine learning models, the data often needs to be cleaned and transformed. This section is titled "Scaling and encoding", which refers to two important steps:

- **Encoding**: Converting categorical (text-based) data into numerical formats. For example, if a column has 'yes'/'no' answers, we might convert them to 1/0.
- **Scaling**: Adjusting the range of values for numerical columns. If one column has values from 0 to 1000 and another from 0 to 1, some machine learning algorithms might treat the column with larger values as more important. Scaling ensures all numerical features contribute equally by putting them on a similar scale (e.g., between 0 and 1, or with a mean of 0 and standard deviation of 1).

There isn't any code in the cell right now, but this is where those steps would typically occur. It's a placeholder for future work.

```
Start coding or generate with AI.
```



Scaling and encoding

5. Principal Component Analysis (PCA)

PCA is a powerful technique used for **dimensionality reduction**. Imagine you have a very complex dataset with many, many features (columns). It can be hard for a human (or even some machine learning models) to understand all these dimensions at once. PCA helps by finding new, fewer features (called "principal components") that capture most of the important information from the original, larger set of features.

- `pca_components_df = pd.DataFrame(pca.components_,columns=df.columns,index = ['PC-1','PC-2'])`: This line (after PCA has been applied, which is currently

missing) would create a table showing what each principal component (PC-1, PC-2, etc.) is made of. Each row represents a principal component, and the values in the columns show how much each original feature contributes to that principal component. For example, if a feature has a high positive number for PC-1, it means that feature is strongly influencing PC-1 in a positive direction.

This table helps us interpret what patterns the principal components are detecting in the data.

```
Start coding or generate with AI.
```

PCA

```
pca_components_df = pd.DataFrame(pca.components_,columns=df.columns,i
pca_components_df
```

	id	age	bp	sg	al	su	rbc	
PC-1	0.289073	-0.129554	-0.130063	0.237258	-0.252492	-0.148418	-0.017660	0.
PC-2	-0.116218	0.181098	0.068397	0.013453	-0.218783	0.102742	0.470618	0.

2 rows × 26 columns

This code block visualizes the PCA component loadings using a heatmap:

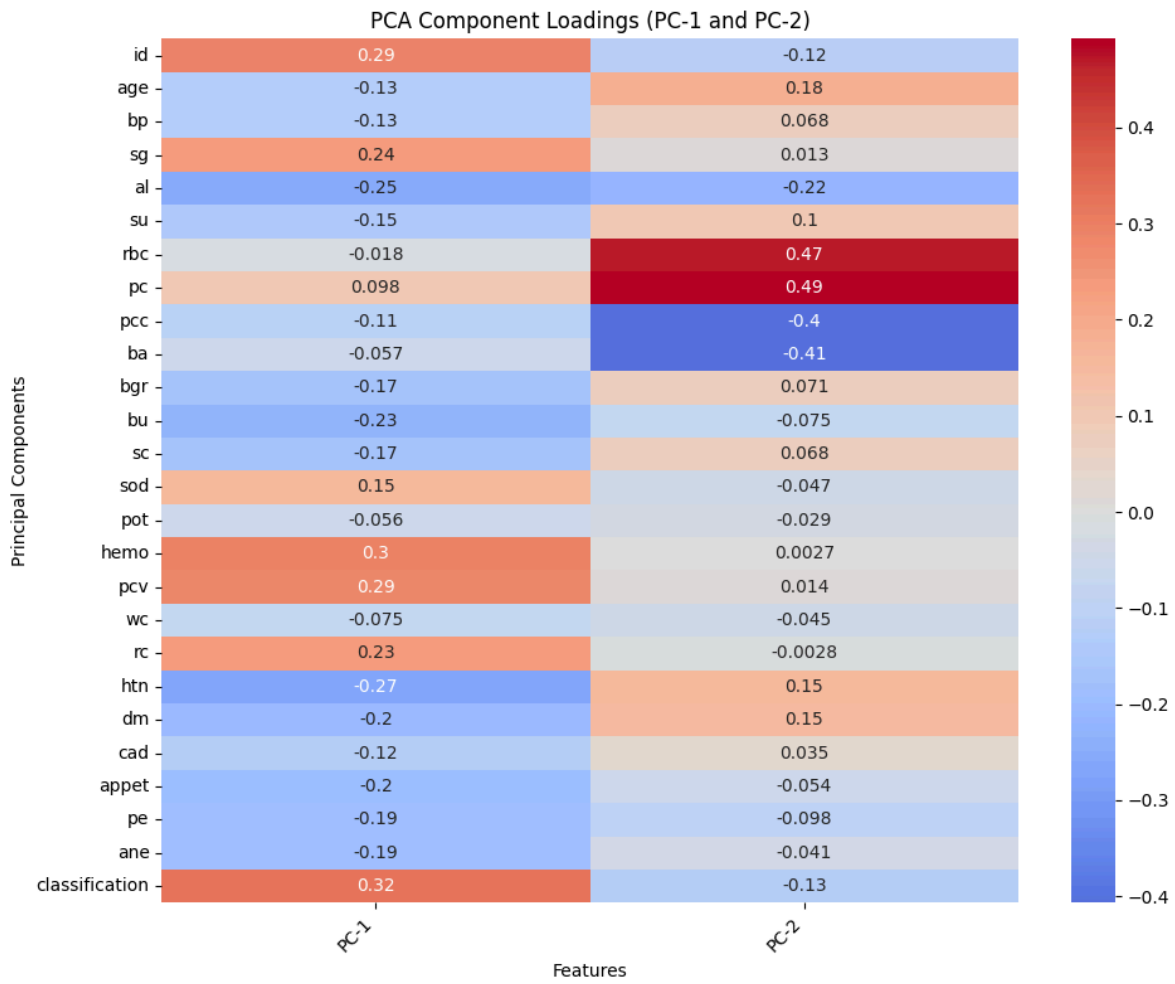
- `plt.figure(figsize=(10, 8))`: Sets the size of the heatmap.
- `sns.heatmap(pca_components_df.T, annot=True, cmap='coolwarm', center=0)`: Creates the heatmap. The `.T` transposes the DataFrame so that features are on one axis and principal components are on the other. `annot=True` shows the actual numbers on the heatmap cells, `cmap='coolwarm'` sets the color scheme (blue for negative contributions, red for positive), and `center=0` ensures that zero is the center of the color scale.
- `plt.title(...)`, `plt.xlabel(...)`, `plt.ylabel(...)`: Set the title and axis labels for the plot.
- `plt.xticks(rotation=45, ha='right')`: Rotates the x-axis labels (feature names) to make them more readable.
- `plt.tight_layout()`: Adjusts the plot to prevent labels from overlapping.
- `plt.show()`: Displays the heatmap.

This heatmap visually shows which original features contribute most strongly (and in which direction, positive or negative) to each principal component. This is crucial for

understanding what each principal component represents in terms of the original data.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
plt.figure(figsize=(10, 8))
sns.heatmap(pca_components_df.T, annot=True, cmap='coolwarm', center=
plt.title("PCA Component Loadings (PC-1 and PC-2)")
plt.xlabel("Features")
plt.ylabel("Principal Components")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



6. Model Training: Building a Predictor

This section begins the process of building machine learning models to classify kidney disease.

- **Logistic Regression**: This is a fundamental machine learning algorithm used for binary classification tasks (like predicting 'yes' or 'no', 'sick' or 'healthy'). Despite its name, it's used for classification, not regression. It calculates the probability

of an event happening. The comment suggests it will be used as a simple baseline model.

The error (`ValueError: Unknown label type: continuous.`) indicates a problem: the `classification` column (our target variable, which should be 'ckd' or 'notckd') was accidentally treated as numerical data, likely during a previous data preprocessing step, and converted into continuous numbers (like 0.5, 1.2, etc.). However, Logistic Regression expects distinct categories (like 0 or 1, or 'ckd' and 'notckd'). This needs to be fixed before the model can be trained correctly. This often means converting categorical text labels to numerical integers (e.g., 'ckd' to 0, 'notckd' to 1) *before* any scaling that might inadvertently turn them into continuous numbers.

```
Start coding or generate with AI.
```

Reasoning: Based on the instructions, since the data is tabular and standard pre-trained models for tabular data are not readily available in common libraries for transfer learning in the traditional sense, the next steps will involve building a model from scratch using standard machine learning techniques. I will use a logistic regression model as a simple baseline classifier for the PCA-transformed features.

```
from sklearn.linear_model import LogisticRegression

# Initialize and train a Logistic Regression model on the PCA-transfo
model = LogisticRegression(random_state=42)
model.fit(X_train, y_train)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipython-input-1006523989.py in <cell line: 0>()
      3 # Initialize and train a Logistic Regression model on the PCA-transformed training data
      4 model = LogisticRegression(random_state=42)
----> 5 model.fit(X_train, y_train)

                              2 frames
/usr/local/lib/python3.12/dist-packages/sklearn/utils/multiclass.py in check_classification_targets(y)
    220         "multilabel-sequences",
    221     ]:
--> 222         raise ValueError(
    223             f"Unknown label type: {y_type}. Maybe you are trying to fit a "
    224             "classifier, which expects discrete classes on a "
```

The error analysis correctly identifies the issue: the target variable (`y_train`) is continuous instead of discrete. This is a common mistake when scaling all numerical